

Ai OPS 2

Harshvardhan Agrawal

September 2026

Q1

Size of the naive docker image is 1.47GB, Multistage: 442MB, reduction of 70.64%

python:3.11-slim doesnt include the full debian build environment that are present in python3:11, resulting in 970MB savings

-no-cache-dir is what caused this difference of 56MB as the wheel pip leaves in the /root/.cache/pip in naive approach while multistage doesnt maintain it.

Slim runtime has no system libgomp but sklearn wheel has its own so it still works.

Q2

One dockerfile cannot make more than one image, so it cannot declare that our spam API needs a seerate redis container. Docker compose solves this by one declarative file, that defines the two services and their shared network, starting order, and configuration, and one docker compose up brings the whole machine up. Compose's scope is still a single docker host, with no scheduling across machines and no automatic reschedulin if the host gives up, whereas kubernetes orchestrates container across clusters, and can place pods by resource reqursts.

Q3

Parallelism chosen is 4, with CPU request = limit = 500m per pod. Out of the 4 CPUs, kube-system pods already request 850m on the control plane node and 100m on the worker, so only about 3.05 CPU is actually schedulable. The validator is single threaded python so 500m is enough, and at most $2 + 3 = 5$ pods of 500m can fit at once, running all 8 together would need 4 CPU which doesnt fit. Any parallelism from 4 to 7 finishes in 2 waves, so 4 is the smallest value that gives this and still leaves around 1 CPU free. Downward API gives NODE_NAME (spec.nodeName) and POD_NAME (metadata.name) to each pod.

`kubectrl get pods -o wide` shows pods 0-3 running at the same time (3 on minikube-m02, 1 on minikube) and then pods 4-7 in the second wave, job completed 8/8 in 68s. Each pod sleeps 30s after validating, otherwise it finishes before the concurrency can be seen. minikube docker driver reports 20 CPUs per node, so the 4 CPU limit is applied through the manifest and not by the scheduler.

Invalid rows per shard (0-7): 25, 15, 12, 8, 30, 23, 14, 16, total 143, all matching the seeded ground truth. These were collected by reading pod logs through the kubernetes API (`read_namespaced_pod_log`). Shared volume was not used because minikube's storage provisioner is `hostPath` based, the PV is just a `ReadWriteOnce` directory on one node, so pods on the other node can't see it. Logs are kept by the kubelet on every node and served by the API server, so they can be read from anywhere.

With 3 nodes of 2 CPU each, I would raise parallelism to 8. Schedulable CPU becomes about 4.95, which fits $2 + 3 + 3 = 8$ pods of 500m, so all shards run in one wave instead of two and the runtime is halved. Going above 8 is useless as completions is 8.

Q4

Deployment has 2 replicas, CPU and memory requests/limits, `readinessProbe` on `/healthz` and `RollingUpdate` with `maxSurge` 1 and `maxUnavailable` 0, exposed with a `NodePort` service.

After deleting pod `...-7jfdn`, a new pod `...-s8ppk` was created within a second and was ready in about 6s. This is done by the `ReplicaSet` controller (inside `kube-controller-manager`, owned by the Deployment), it keeps comparing the number of pods matching its selector with the desired replicas: 2, and when it dropped to 1 it created a new pod from the template.

For v2, `/healthz` also returns the version string. rollout status showed successfully rolled out and rollout history shows 2 revisions. While a client was hitting the service every 0.2s, 0 out of 277 requests failed (58 served by v1 and then 219 by v2). If this was a Job instead, it would never complete because `uvicorn` never exits, a crash would only be retried till `backoffLimit`, and since the pod template of a Job is immutable, moving to v2 would need deleting and recreating the job, which means downtime.

A serving API is long running and success means N healthy replicas at all times, so self healing and zero downtime updates of a Deployment fit it. The Q3 validation is batch work where success means each index exits 0 once, so a Job with completions, parallelism and retries fits it, and restarting already finished pods would be wrong.